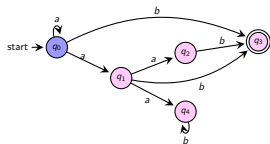


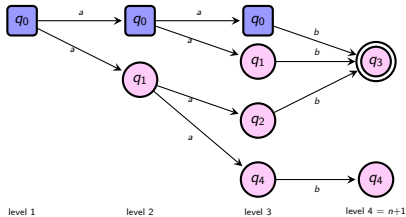
CS 310 : Automata Theory

S. Krishna

A two player graph game



automaton A



$G(A, w)$ for $w = aab$

Strategies and Plays

Plays

A *play* of $G(A, w)$ is a sequence $(q_1, 1), (q_2, 2), \dots, (q_{n+1}, n+1)$ with q_1 = initial state and q_{i+1} some state in $\delta(q_i, w_i)$ for each i .

Strategies and Plays

Plays

A *play* of $G(A, w)$ is a sequence $(q_1, 1), (q_2, 2), \dots, (q_{n+1}, n+1)$ with q_1 = initial state and q_{i+1} some state in $\delta(q_i, w_i)$ for each i .

Strategies

A (positional) *strategy* σ_{Eve} (respectively σ_{Adam}) for player Eve (respectively Adam) fixes, at each vertex *that player owns*, which outgoing edge to take.

Strategies and Plays

Plays

A *play* of $G(A, w)$ is a sequence $(q_1, 1), (q_2, 2), \dots, (q_{n+1}, n+1)$ with q_1 = initial state and q_{i+1} some state in $\delta(q_i, w_i)$ for each i .

Strategies

A (positional) *strategy* σ_{Eve} (respectively σ_{Adam}) for player Eve (respectively Adam) fixes, at each vertex *that player owns*, which outgoing edge to take.

- A play is *consistent with* σ if, at every vertex σ is defined on, the play follows σ 's choice.

Strategies and Plays

Plays

A *play* of $G(A, w)$ is a sequence $(q_1, 1), (q_2, 2), \dots, (q_{n+1}, n+1)$ with q_1 = initial state and q_{i+1} some state in $\delta(q_i, w_i)$ for each i .

Strategies

A (positional) *strategy* σ_{Eve} (respectively σ_{Adam}) for player Eve (respectively Adam) fixes, at each vertex *that player owns*, which outgoing edge to take.

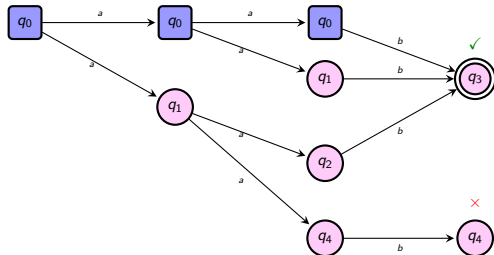
- ▶ A play is *consistent with* σ if, at every vertex σ is defined on, the play follows σ 's choice.
- ▶ σ_{Eve} is a **winning strategy** for Eve if **every** play consistent with σ_{Eve} ends at some $(q, n+1)$ with $q \in F$ irrespective of σ_{Adam} .

To Prove :

Eve has a winning strategy in $G(A, w) \iff A$ accepts w .

Backward reachability, level by level

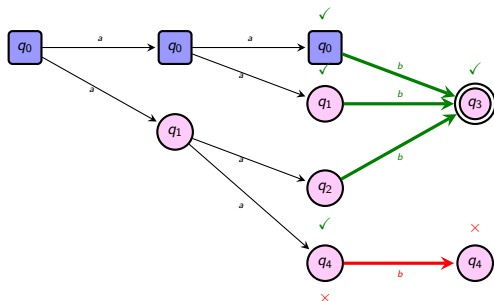
Mark winning states for Eve



- Level $n+1 = 4$ is terminal: win iff in $F = \{q_3\}$. So $(q_3, 4)$ wins, $(q_4, 4)$ loses.

Backward reachability, level by level

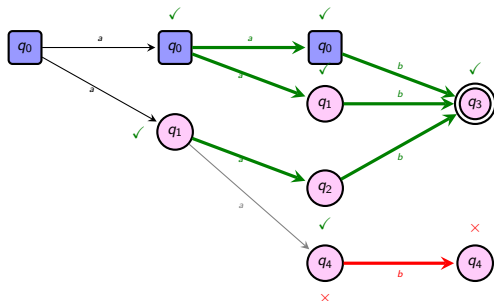
Mark winning states for Eve



- ▶ Level $n+1 = 4$ is terminal: win iff in $F = \{q_3\}$. So $(q_3, 4)$ wins, $(q_4, 4)$ loses.
- ▶ Level 3: $(q_0, 3)$, $(q_1, 3)$, $(q_2, 3)$ each have one child, q_3 (wins) so they win. $(q_4, 3)$'s only child is $(q_4, 4)$ (loses), it *loses*.

Backward reachability, level by level

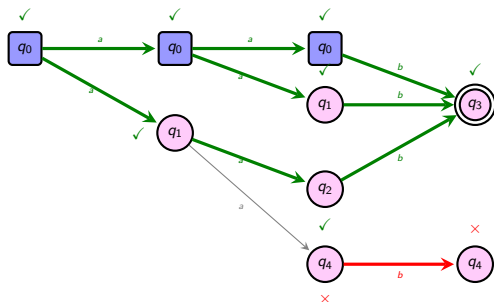
Mark winning states for Eve



- ▶ Level $n+1 = 4$ is terminal: win iff in $F = \{q_3\}$. So $(q_3, 4)$ wins, $(q_4, 4)$ loses.
- ▶ Level 3: $(q_0, 3)$, $(q_1, 3)$, $(q_2, 3)$ each have one child, q_3 (wins) so they win. $(q_4, 3)$'s only child is $(q_4, 4)$ (loses), it *loses*.
- ▶ Level 2: $(q_0, 2)$ is $\forall$ -owned, has *both* children winning. $(q_1, 2)$ is $\exists$ -owned with *two* choices, q_2 (win) and q_4 (lose), it picks q_2 and *avoids* q_4 : wins.

Backward reachability, level by level

Mark winning states for Eve

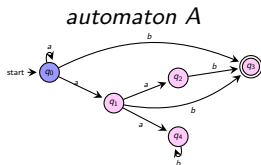


- ▶ Level $n+1 = 4$ is terminal: win iff in $F = \{q_3\}$. So $(q_3, 4)$ wins, $(q_4, 4)$ loses.
- ▶ Level 3: $(q_0, 3)$, $(q_1, 3)$, $(q_2, 3)$ each have one child, q_3 (wins) so they win. $(q_4, 3)$'s only child is $(q_4, 4)$ (loses), it *loses*.
- ▶ Level 2: $(q_0, 2)$ is $\forall$ -owned, has *both* children winning. $(q_1, 2)$ is $\exists$ -owned with *two* choices, q_2 (win) and q_4 (lose), it picks q_2 and *avoids* q_4 : wins.
- ▶ Level 1: $(q_0, 1)$ has *both* children winning $\Rightarrow$ Eve wins from the start $\Rightarrow A$ accepts aab .

To Prove :

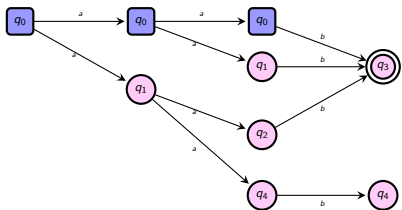
Eve has a winning strategy in $G(A, w) \iff A$ accepts w .

Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$



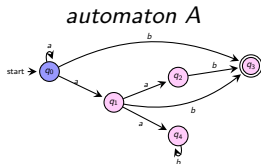
accepting run tree, $w = aab$

q_0

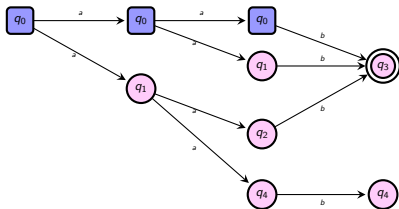
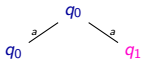


The complete game graph on $w = aab$.

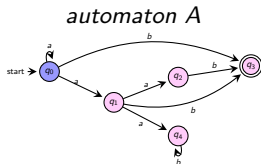
Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$



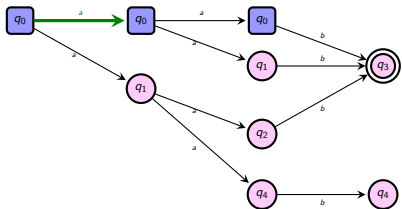
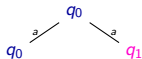
accepting run tree, $w = aab$



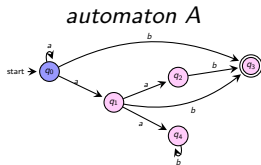
Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$



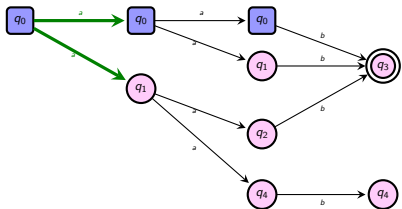
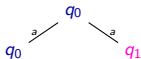
accepting run tree, $w = aab$



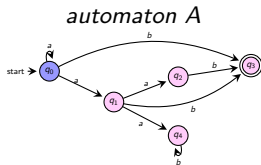
Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$



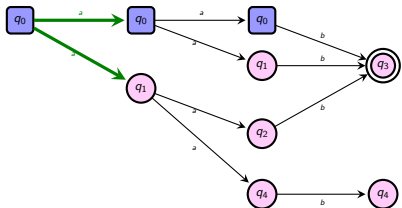
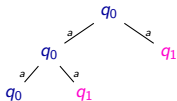
accepting run tree, $w = aab$



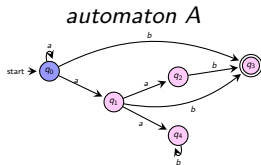
Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$



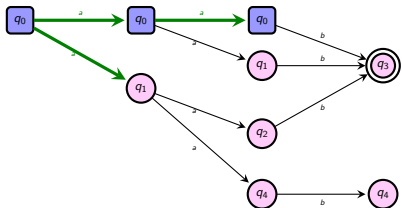
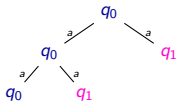
accepting run tree, $w = aab$



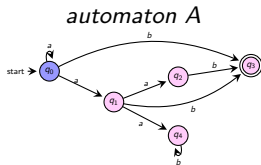
Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$



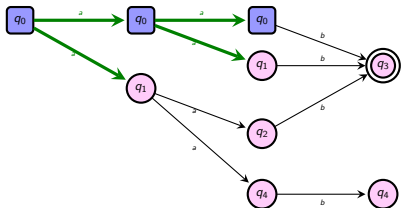
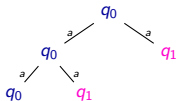
accepting run tree, $w = aab$



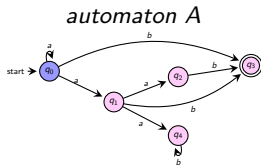
Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$



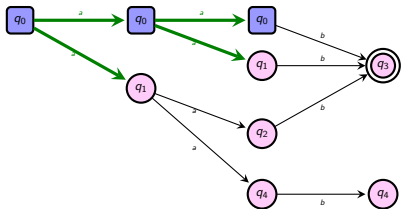
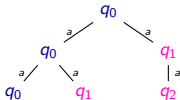
accepting run tree, $w = aab$



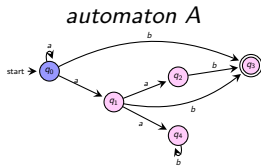
Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$



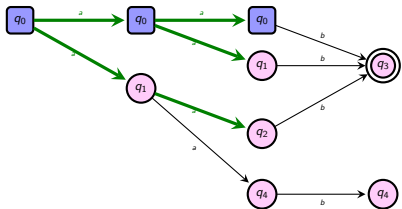
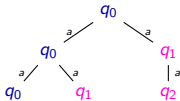
accepting run tree, $w = aab$



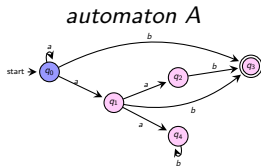
Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$



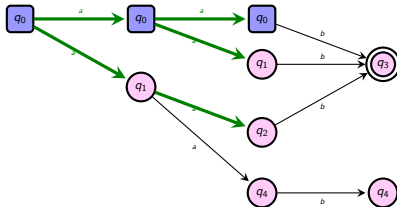
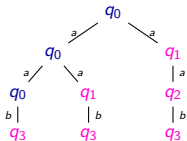
accepting run tree, $w = aab$



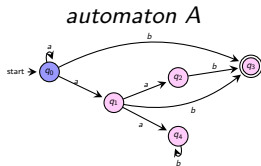
Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$



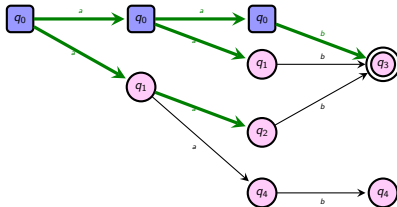
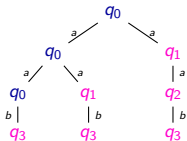
accepting run tree, $w = aab$



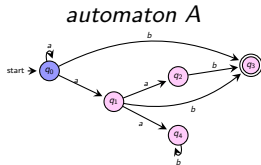
Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$



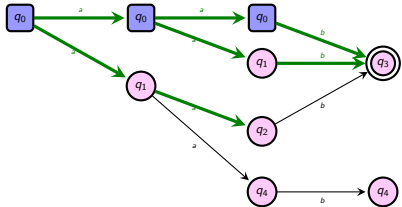
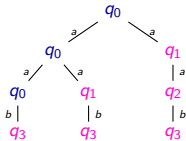
accepting run tree, $w = aab$



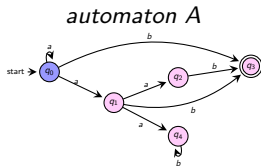
Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$



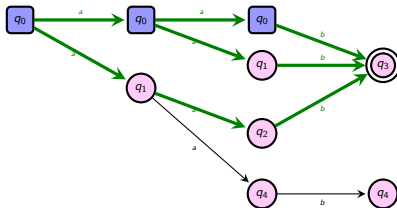
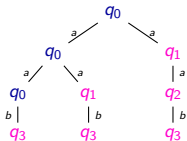
accepting run tree, $w = aab$



Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$



accepting run tree, $w = aab$



Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$

If A accepts w , Eve has a (positional) winning strategy in $G(A, w)$.

Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$

If A accepts w , Eve has a (positional) winning strategy in $G(A, w)$.

- ▶ A accepts $w \Rightarrow$ fix an accepting run tree T : every $\forall$ -node's children are *all* of $\delta(q, a)$; every $\exists$ -node has *one* chosen child; every leaf is in F .

Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$

If A accepts w , Eve has a (positional) winning strategy in $G(A, w)$.

- ▶ A accepts $w \Rightarrow$ fix an accepting run tree T : every $\forall$ -node's children are *all* of $\delta(q, a)$; every $\exists$ -node has *one* chosen child; every leaf is in F .
- ▶ **Eve's Strategy**: at any $\exists$ -owned vertex the play reaches in $G(A, w)$, follow T 's chosen child.

Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$

If A accepts w , Eve has a (positional) winning strategy in $G(A, w)$.

- ▶ A accepts $w \Rightarrow$ fix an accepting run tree T : every $\forall$ -node's children are *all* of $\delta(q, a)$; every $\exists$ -node has *one* chosen child; every leaf is in F .
- ▶ **Eve's Strategy**: at any $\exists$ -owned vertex the play reaches in $G(A, w)$, follow T 's chosen child.
- ▶ **Why Eve wins**: at a $\forall$ -owned vertex, T lists every $\delta(q, a)$ -successor as a child, so whichever edge Adam takes in $G(A, w)$, the play stays inside T .

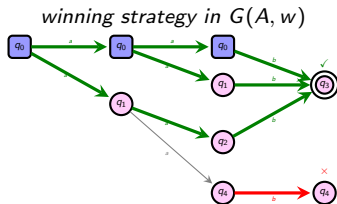
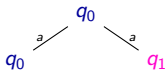
Acceptance of w in $A \Rightarrow$ Eve wins in $G(A, w)$

If A accepts w , Eve has a (positional) winning strategy in $G(A, w)$.

- ▶ A accepts $w \Rightarrow$ fix an accepting run tree T : every $\forall$ -node's children are *all* of $\delta(q, a)$; every $\exists$ -node has *one* chosen child; every leaf is in F .
- ▶ **Eve's Strategy**: at any $\exists$ -owned vertex the play reaches in $G(A, w)$, follow T 's chosen child.
- ▶ **Why Eve wins**: at a $\forall$ -owned vertex, T lists *every* $\delta(q, a)$ -successor as a child, so whichever edge Adam takes in $G(A, w)$, the play stays inside T .
- ▶ The play can never leave T , so it reaches a leaf of T at level $n+1$ which is in F . Eve wins, irrespective of Adam's choices.

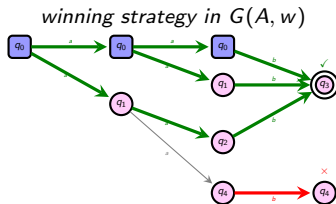
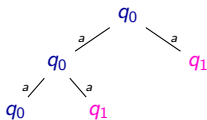
Eve wins in $G(A, w) \Rightarrow A$ accepts w

Reading the strategy off the backward computation:



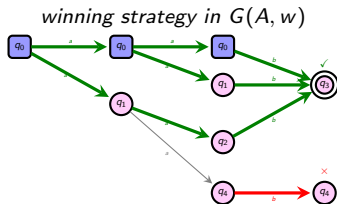
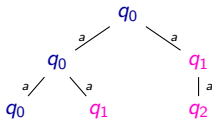
Eve wins in $G(A, w) \Rightarrow A$ accepts w

Reading the strategy off the backward computation:



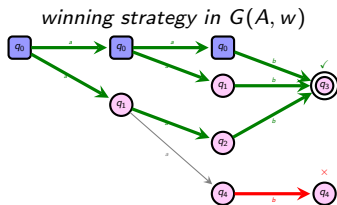
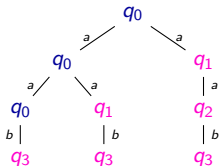
Eve wins in $G(A, w) \Rightarrow A$ accepts w

Reading the strategy off the backward computation:



Eve wins in $G(A, w) \Rightarrow A$ accepts w

Reading the strategy off the backward computation:



Eve wins in $G(A, w) \Rightarrow A$ accepts w

If Eve has a winning strategy in $G(A, w)$ from (q, i) , there is an accepting run tree of A on $w_i \cdots w_n$ starting at q .

Eve wins in $G(A, w) \Rightarrow A$ accepts w

If Eve has a winning strategy in $G(A, w)$ from (q, i) , there is an accepting run tree of A on $w_i \cdots w_n$ starting at q .

Proof, by induction on $n + 1 - i$:

- **Base case** $i = n+1$: winning means $q \in F$; the one-node tree q is trivially accepting.

Eve wins in $G(A, w) \Rightarrow A$ accepts w

If Eve has a winning strategy in $G(A, w)$ from (q, i) , there is an accepting run tree of A on $w_i \cdots w_n$ starting at q .

Proof, by induction on $n + 1 - i$:

- ▶ **Base case** $i = n+1$: winning means $q \in F$; the one-node tree q is trivially accepting.
- ▶ **Step**, $q \in Q_\exists$: the strategy chooses a winning child $(q', i+1)$; by induction there is an accepting run tree of A rooted at q' for $w_{i+1} \cdots w_n$.

Eve wins in $G(A, w) \Rightarrow A$ accepts w

If Eve has a winning strategy in $G(A, w)$ from (q, i) , there is an accepting run tree of A on $w_i \cdots w_n$ starting at q .

Proof, by induction on $n + 1 - i$:

- ▶ **Base case** $i = n+1$: winning means $q \in F$; the one-node tree q is trivially accepting.
- ▶ **Step**, $q \in Q_\exists$: the strategy chooses a winning child $(q', i+1)$; by induction there is an accepting run tree of A rooted at q' for $w_{i+1} \cdots w_n$. Extend the tree making this the child of q . The new tree is rooted at q is an accepting run tree for $w_i \cdots w_n$.

Eve wins in $G(A, w) \Rightarrow A$ accepts w

If Eve has a winning strategy in $G(A, w)$ from (q, i) , there is an accepting run tree of A on $w_i \cdots w_n$ starting at q .

Proof, by induction on $n + 1 - i$:

- ▶ **Base case** $i = n+1$: winning means $q \in F$; the one-node tree q is trivially accepting.
- ▶ **Step, $q \in Q_\exists$** : the strategy chooses a winning child $(q', i+1)$; by induction there is an accepting run tree of A rooted at q' for $w_{i+1} \cdots w_n$. Extend the tree making this the child of q . The new tree is rooted at q is an accepting run tree for $w_i \cdots w_n$.
- ▶ **Step, $q \in Q_\forall$** : winning forces every child q' at level $i + 1$ to win;

Eve wins in $G(A, w) \Rightarrow A$ accepts w

If Eve has a winning strategy in $G(A, w)$ from (q, i) , there is an accepting run tree of A on $w_i \cdots w_n$ starting at q .

Proof, by induction on $n + 1 - i$:

- ▶ **Base case** $i = n+1$: winning means $q \in F$; the one-node tree q is trivially accepting.
- ▶ **Step, $q \in Q_{\exists}$** : the strategy chooses a winning child $(q', i+1)$; by induction there is an accepting run tree of A rooted at q' for $w_{i+1} \cdots w_n$. Extend the tree making this the child of q . The new tree is rooted at q is an accepting run tree for $w_i \cdots w_n$.
- ▶ **Step, $q \in Q_{\forall}$** : winning forces every child q' at level $i + 1$ to win; by induction each run tree of A rooted at each such q' is accepting for $w_{i+1} \cdots w_n$.

Eve wins in $G(A, w) \Rightarrow A$ accepts w

If Eve has a winning strategy in $G(A, w)$ from (q, i) , there is an accepting run tree of A on $w_i \cdots w_n$ starting at q .

Proof, by induction on $n + 1 - i$:

- ▶ **Base case** $i = n+1$: winning means $q \in F$; the one-node tree q is trivially accepting.
- ▶ **Step, $q \in Q_\exists$** : the strategy chooses a winning child $(q', i+1)$; by induction there is an accepting run tree of A rooted at q' for $w_{i+1} \cdots w_n$. Extend the tree making this the child of q . The new tree is rooted at q is an accepting run tree for $w_i \cdots w_n$.
- ▶ **Step, $q \in Q_\forall$** : winning forces every child q' at level $i + 1$ to win; by induction each run tree of A rooted at each such q' is accepting for $w_{i+1} \cdots w_n$. Extend the tree by attaching all of these q' as children of q . This is then an accepting run tree rooted at q for $w_i \cdots w_n$.

Eve wins in $G(A, w) \Rightarrow A$ accepts w

If Eve has a winning strategy in $G(A, w)$ from (q, i) , there is an accepting run tree of A on $w_i \cdots w_n$ starting at q .

Proof, by induction on $n + 1 - i$:

- ▶ **Base case** $i = n+1$: winning means $q \in F$; the one-node tree q is trivially accepting.
- ▶ **Step, $q \in Q_\exists$** : the strategy chooses a winning child $(q', i+1)$; by induction there is an accepting run tree of A rooted at q' for $w_{i+1} \cdots w_n$. Extend the tree making this the child of q . The new tree is rooted at q is an accepting run tree for $w_i \cdots w_n$.
- ▶ **Step, $q \in Q_\forall$** : winning forces every child q' at level $i + 1$ to win; by induction each run tree of A rooted at each such q' is accepting for $w_{i+1} \cdots w_n$. Extend the tree by attaching all of these q' as children of q . This is then an accepting run tree rooted at q for $w_i \cdots w_n$.
- ▶ At $(q_0, 1)$: Eve wins $\Rightarrow$ an accepting run of A for $w_1 \cdots w_n$ exists.

The game is determined

Positional determinacy

In every $G(A, w)$, exactly one of Eve, Adam has a winning strategy.

The game is determined

Positional determinacy

In every $G(A, w)$, exactly one of Eve, Adam has a winning strategy.

- ▶ Straight forward from backward induction: $\text{win}(q, n+1)$ if $q \in F$, and $\text{win}(q, i)$ is an $\wedge$ or $\vee$ over its children, depending on who owns q .
- ▶ By induction on $n+1 - i$, it assigns a Boolean to every vertex

The game is determined

Positional determinacy

In every $G(A, w)$, exactly one of Eve, Adam has a winning strategy.

- ▶ Straight forward from backward induction: $\text{win}(q, n+1)$ if $q \in F$, and $\text{win}(q, i)$ is an $\bigwedge$ or $\bigvee$ over its children, depending on who owns q .
- ▶ By induction on $n+1 - i$, it assigns a Boolean to every vertex
- ▶ Whoever's value is true at $(s, 1)$ has a winning strategy; so *some* player wins, and not both. Hence exactly one does.

Complementation = Swapping players + final states

Given $A = (Q_{\exists}, Q_{\forall}, \Sigma, \delta, s, F)$, let $\bar{A} = (Q_{\forall}, Q_{\exists}, \Sigma, \delta, s, Q \setminus F)$. Then $L(\bar{A}) = \Sigma^* \setminus L(A)$.

Complementation = Swapping players + final states

Given $A = (Q_{\exists}, Q_{\forall}, \Sigma, \delta, s, F)$, let $\bar{A} = (Q_{\forall}, Q_{\exists}, \Sigma, \delta, s, Q \setminus F)$. Then $L(\bar{A}) = \Sigma^* \setminus L(A)$.

- Swapping $Q_{\exists} \leftrightarrow Q_{\forall}$ swaps *who moves where*: Eve in $G(\bar{A}, w)$ moves exactly at the vertices Adam moved at in $G(A, w)$, and vice versa.

Complementation = Swapping players + final states

Given $A = (Q_{\exists}, Q_{\forall}, \Sigma, \delta, s, F)$, let $\bar{A} = (Q_{\forall}, Q_{\exists}, \Sigma, \delta, s, Q \setminus F)$. Then $L(\bar{A}) = \Sigma^* \setminus L(A)$.

- ▶ Swapping $Q_{\exists} \leftrightarrow Q_{\forall}$ swaps *who moves where*: Eve in $G(\bar{A}, w)$ moves exactly at the vertices Adam moved at in $G(A, w)$, and vice versa.
- ▶ Complementing F swaps the *target to be reached*: Eve in $G(\bar{A}, w)$ wins by reaching $\bar{F} = Q \setminus F$, which is Adam's old target in $G(A, w)$.

Complementation = Swapping players + final states

Given $A = (Q_{\exists}, Q_{\forall}, \Sigma, \delta, s, F)$, let $\bar{A} = (Q_{\forall}, Q_{\exists}, \Sigma, \delta, s, Q \setminus F)$. Then $L(\bar{A}) = \Sigma^* \setminus L(A)$.

- ▶ Swapping $Q_{\exists} \leftrightarrow Q_{\forall}$ swaps *who moves where*: Eve in $G(\bar{A}, w)$ moves exactly at the vertices Adam moved at in $G(A, w)$, and vice versa.
- ▶ Complementing F swaps the *target to be reached*: Eve in $G(\bar{A}, w)$ wins by reaching $\bar{F} = Q \setminus F$, which is Adam's old target in $G(A, w)$.
- ▶ At each level, a vertex is winning for Eve in $G(\bar{A}, w)$ iff it was losing for Eve in $G(A, w)$:
base case ($i = n+1$): $q \in \bar{F} \iff q \notin F$; step: swapping $\exists/\forall$ swaps $\vee$ for $\wedge$ in the backward recursion.

Complementation = Swapping players + final states

Given $A = (Q_{\exists}, Q_{\forall}, \Sigma, \delta, s, F)$, let $\bar{A} = (Q_{\forall}, Q_{\exists}, \Sigma, \delta, s, Q \setminus F)$. Then $L(\bar{A}) = \Sigma^* \setminus L(A)$.

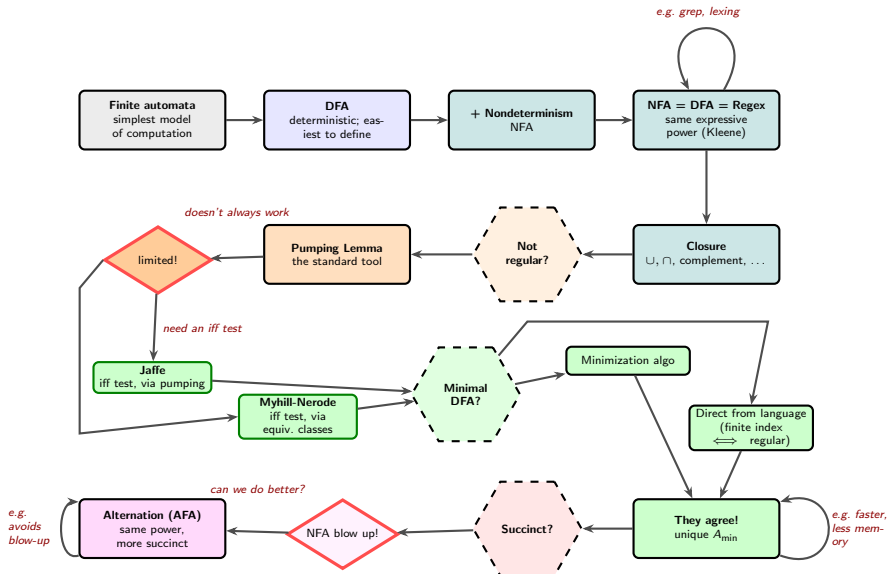
- ▶ Swapping $Q_{\exists} \leftrightarrow Q_{\forall}$ swaps *who moves where*: Eve in $G(\bar{A}, w)$ moves exactly at the vertices Adam moved at in $G(A, w)$, and vice versa.
- ▶ Complementing F swaps the *target to be reached*: Eve in $G(\bar{A}, w)$ wins by reaching $\bar{F} = Q \setminus F$, which is Adam's old target in $G(A, w)$.
- ▶ At each level, a vertex is winning for Eve in $G(\bar{A}, w)$ iff it was losing for Eve in $G(A, w)$:
base case ($i = n+1$): $q \in \bar{F} \iff q \notin F$; step: swapping $\exists/\forall$ swaps $\vee$ for $\wedge$ in the backward recursion.
- ▶ So, at the start: Eve wins $G(\bar{A}, w) \iff$ Eve does *not* win $G(A, w)$.

Complementation = Swapping players + final states

Given $A = (Q_{\exists}, Q_{\forall}, \Sigma, \delta, s, F)$, let $\bar{A} = (Q_{\forall}, Q_{\exists}, \Sigma, \delta, s, Q \setminus F)$. Then $L(\bar{A}) = \Sigma^* \setminus L(A)$.

- ▶ Swapping $Q_{\exists} \leftrightarrow Q_{\forall}$ swaps *who moves where*: Eve in $G(\bar{A}, w)$ moves exactly at the vertices Adam moved at in $G(A, w)$, and vice versa.
- ▶ Complementing F swaps the *target to be reached*: Eve in $G(\bar{A}, w)$ wins by reaching $\bar{F} = Q \setminus F$, which is Adam's old target in $G(A, w)$.
- ▶ At each level, a vertex is winning for Eve in $G(\bar{A}, w)$ iff it was losing for Eve in $G(A, w)$:
base case ($i = n+1$): $q \in \bar{F} \iff q \notin F$; step: swapping $\exists/\forall$ swaps $\vee$ for $\wedge$ in the backward recursion.
- ▶ So, at the start: Eve wins $G(\bar{A}, w) \iff$ Eve does *not* win $G(A, w)$.
- ▶ By determinacy, **Eve does not win $G(A, w) \iff$ Adam wins $G(A, w)$** . Combined with the earlier result connecting Eve's winning and acceptance (applied to A and to $\bar{A}$): $\bar{A}$ accepts $w \iff A$ does not accept w .

Module 1: Models with finite memory, so far



Wrapping Up : Extensions and Applications

Finite-state Transducers

A DFA that also *writes* something as it reads.

Finite-state Transducers

A DFA that also *writes* something as it reads.

A finite state transducer is a tuple $T = (Q, \Sigma, \Gamma, \delta, q_0)$ where $\delta : Q \times \Sigma \rightarrow Q \times \Gamma^*$.

- ▶ Same structure as a DFA: states, an alphabet Σ to read.
- ▶ An output alphabet Γ , each transition *emits* a word over Γ .
- ▶ Read an input word $w \in \Sigma^*$ letter by letter, the output is obtained by concatenating the outputs along the way.

Examples: English pluralization

The task

Given a singular noun, output its plural.

Examples: English pluralization

The task

Given a singular noun, output its plural.

Singular	Plural
cat	cats
fox	foxes
city	cities
goose	geese
...	...

Examples: English pluralization

The task

Given a singular noun, output its plural.

Singular	Plural
cat	cats
fox	foxes
city	cities
goose	geese
...	...

- One option: store every pair in a table.

Examples: English pluralization

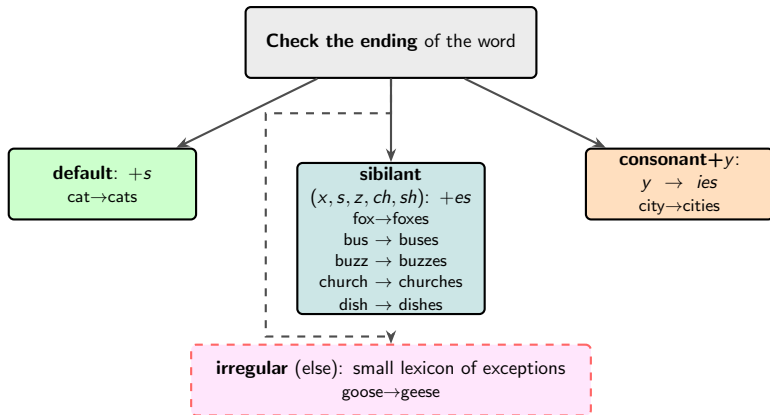
The task

Given a singular noun, output its plural.

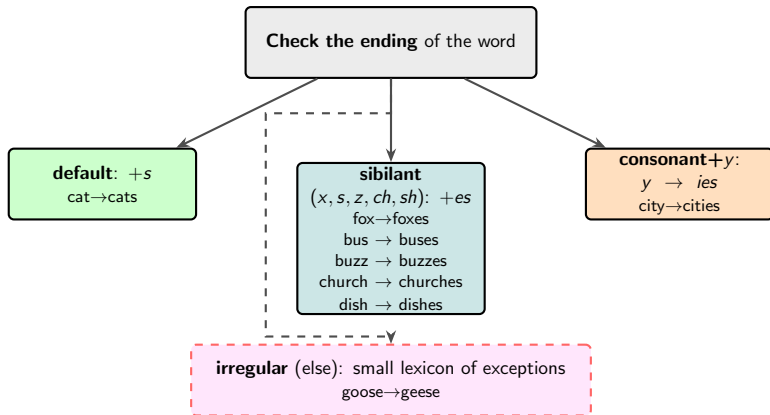
Singular	Plural
cat	cats
fox	foxes
city	cities
goose	geese
...	...

- ▶ One option: store every pair in a table.
- ▶ **Problem:** the table grows with the vocabulary, and says nothing about a brand-new word.

Encode the rules, not the table



Encode the rules, not the table



A handful of rules plus a short exception replaces an unbounded table : this *is* a transducer.

Applications, and why we minimize

Where FSTs show up in NLP

- ▶ **Text processing:** e.g. capitalization, pluralization, tokenizing.
- ▶ **Translation:** input in one language, output in another.
- ▶ **Speech processing:** input a sound pattern, output an English phrase.

Build one large transducer covering all the rules and the lexicon, then *minimize* it.

Applications, and why we minimize

Where FSTs show up in NLP

- ▶ **Text processing:** e.g. capitalization, pluralization, tokenizing.
- ▶ **Translation:** input in one language, output in another.
- ▶ **Speech processing:** input a sound pattern, output an English phrase.

Build one large transducer covering all the rules and the lexicon, then *minimize* it.

- ▶ Fewer states = faster lookups, less memory, at the scale of an entire language's vocabulary.

Applications, and why we minimize

Where FSTs show up in NLP

- ▶ **Text processing:** e.g. capitalization, pluralization, tokenizing.
- ▶ **Translation:** input in one language, output in another.
- ▶ **Speech processing:** input a sound pattern, output an English phrase.

Build one large transducer covering all the rules and the lexicon, then *minimize* it.

- ▶ Fewer states = faster lookups, less memory, at the scale of an entire language's vocabulary.
- ▶ Speech and language processing toolkit : **OpenFST** implements determinization, minimization and many other operations on FSTs.
- ▶ See Piazza for a paper on **Transducers in Language and Speech**
- ▶ End of Module 1